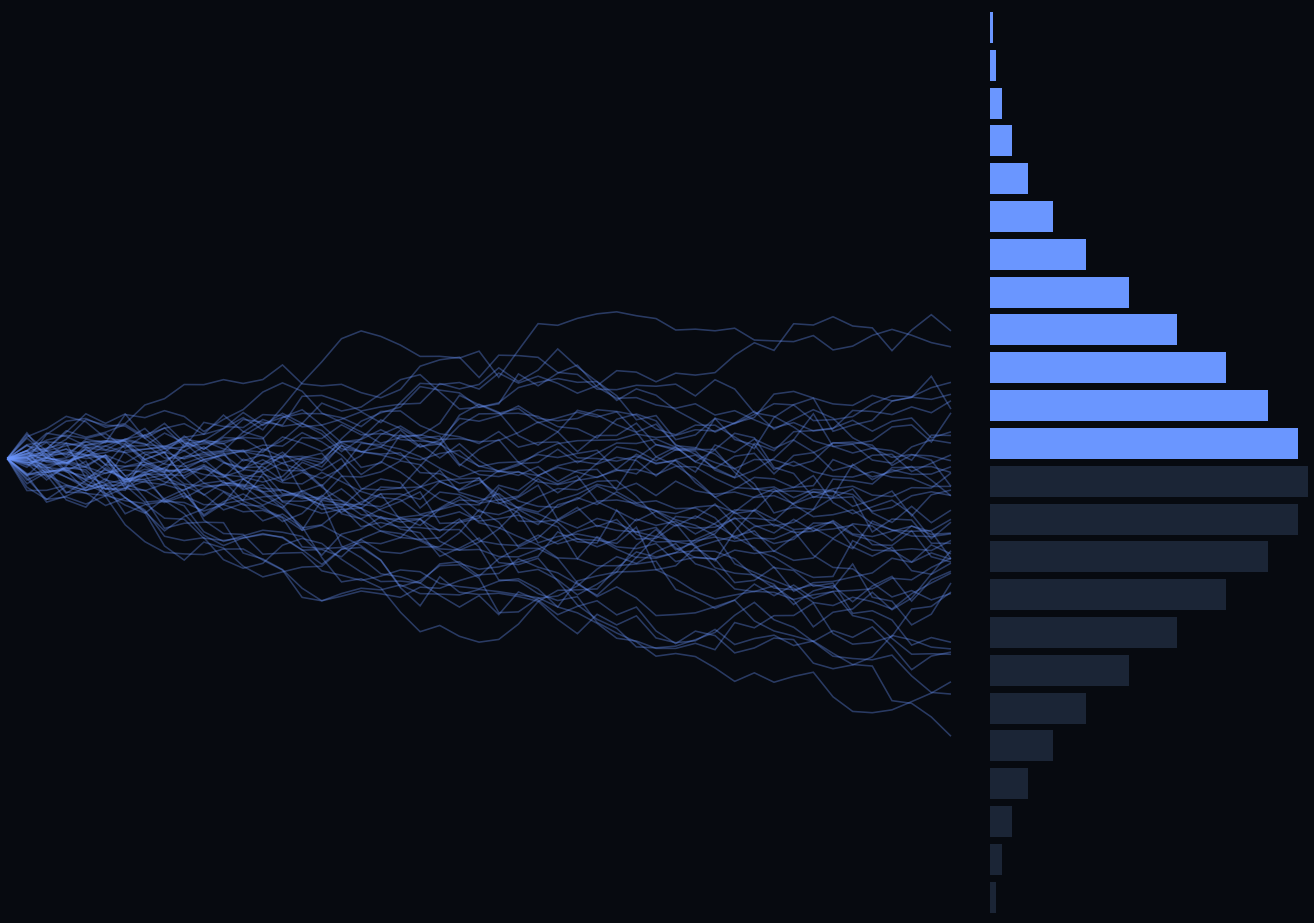
Rolletto

The Complete Guide

How to use the tool · The exact valuation methods · How the code works



Probabilistic DCF and LBO valuation powered by Monte Carlo simulation · 08 November 2026

Educational purposes only — not investment advice.

Contents

Contents 2

About this guide 3

Part A — Using Rolletto 4

A1. What Rolletto does . . . 4
A2. Running it . . . 4
A3. Finding a company . . . 5
A4. The analysis workspace . . . 5
A5. The result tabs . . . 7
A6. Financial companies . . . 12
A7. Exports, appearance and formats . . . 13
A8. Reading the results well . . . 15

Part B — Valuation methods 16

B1. Data and cleaning . . . 16
B2. Default assumptions . . . 16
B3. Cost of capital . . . 17
B4. Discounted cash flow . . . 17
B5. Leveraged buyout . . . 19
B6. Monte Carlo simulation . . . 20
B7. Comparable companies and the football field . . . 22
B8. LBO suitability score . . . 22
B9. Financial companies . . . 22
B10. Hand-checked example — Acme Corp . . . 23

Part C — How the code works 24

C1. Architecture . . . 24
C2. The life of a valuation request . . . 24
C3. Backend — data layer . . . 25
C4. Backend — the vectorised engine . . . 26
C5. Backend — assumptions and overrides . . . 28
C6. Backend — API and errors . . . 29
C7. Frontend . . . 29
C8. Exports . . . 31
C9. Testing . . . 31
C10. Extending Rolletto . . . 32
C11. Limitations . . . 32
Glossary . . . 32

About this guide

This guide explains Rolletto in three parts. **Part A** is a user manual: what every screen, control and number means and how to read the results. **Part B** documents the valuation methods exactly as implemented — every formula, default and convention — with a hand-checked example you can verify on paper. **Part C** walks through the code: the architecture, how a request flows through the system, and annotated excerpts of the key functions, taken directly from the source files.

Live figures in this guide (The Coca-Cola Company, ticker KO) were produced by the running app on 08 November 2026 with default assumptions, 10,000 simulations and seed 42. They will differ when you run it, because market data changes daily.

If you want to…	Read
Run the app and value a company	A1 – A3
Understand a specific tab, chart or number	A4 – A7
Know exactly how a figure is calculated	Part B
Check the maths by hand	B10 (Acme worked example)
Change or extend the code	Part C

Part A — Using Rolletto

A1. What Rolletto does

Rolletto values a public company the way an investment-banking analyst would — with a discounted cash flow (DCF) model and a leveraged buyout (LBO) model — and then asks how uncertain that answer is. Instead of one number built on a dozen inputs nobody knows precisely, every key input is a probability distribution. Rolletto draws 10,000 correlated sets of inputs, runs the full DCF and LBO for each, and reports the *distribution* of outcomes.

That lets it answer better questions than "what is it worth?":

- How likely is the company to be worth more than today's share price?
- If a private-equity sponsor bought it today, what are the odds of earning a 20% return? Of losing money?
- Which assumption moves the value most?
- How does the DCF range compare with trading comparables and the 52-week trading range?

The name comes from the roulette wheel: Monte Carlo simulation is named after the casino in Monaco, and each simulation is one spin — one plausible future for the company.

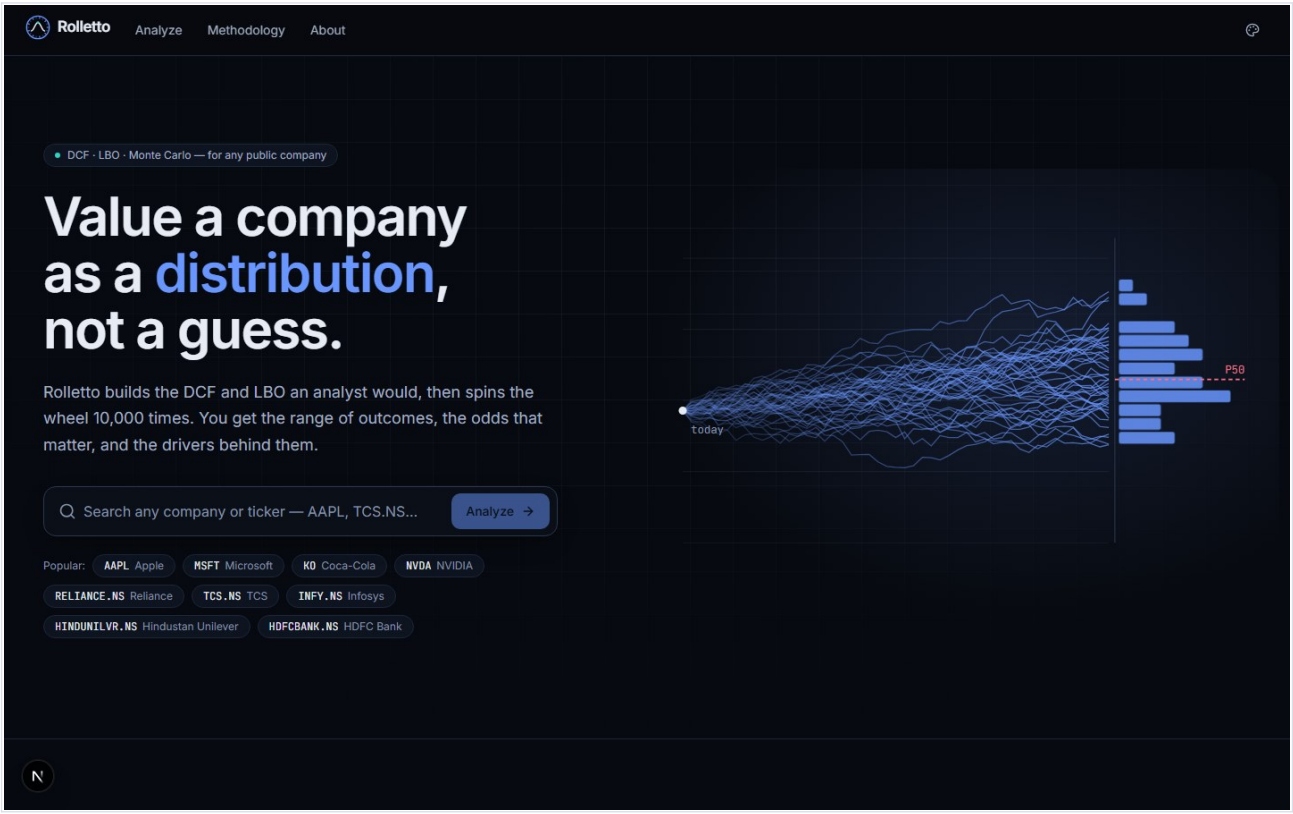


Figure A1 — Home page: search any company, or start from a popular ticker (US and Indian listings).

A2. Running it

Rolletto has two parts that run side by side: a Python **backend** (FastAPI, on port 8000) that fetches data and does all the maths, and a **frontend** website (Next.js, on port 3000) that you use in the browser.

Step	Command (run in the rolletto folder)	What it does
1. Install (once)	`npm run setup`	Installs the website packages, creates backend/.venv and installs the Python packages.

Step	Command (run in the rolletto folder)	What it does
2. Start	`npm run dev`	Starts the API on http://localhost:8000 and the site on http://localhost:3000.
3. Open	`http://localhost:3000`	Search a ticker, or open /analyze/AAPL directly.
Tests	`npm test`	Runs the 65 backend tests.
Checks	`npm run lint`	ruff, ESLint and a strict TypeScript check.

> **Tip** · If your global pip configuration points at an unreachable package index, install with node `scripts/py.mjs -m pip install --isolated -r requirements-dev.txt`.

The API's interactive documentation is at http://localhost:8000/docs — every endpoint can be tried from there.

A3. Finding a company

Type a company name or ticker in the search box on the home page or the navigation bar. Suggestions appear as you type; use the arrow keys and Enter, or click one. Pressing Enter with no suggestion selected opens whatever you typed as a ticker.

Market	Ticker format	Examples
United States	Plain symbol	`AAPL, MSFT, KO, NVDA`
India — NSE	Symbol + .NS	`RELIANCE.NS, TCS.NS, HDFCBANK.NS`
India — BSE	Symbol + .BO	`500325.BO`
Other exchanges	Yahoo Finance suffix	`.L (London), .T (Tokyo), .HK (Hong Kong), .DE (Xetra)`

If a ticker does not exist you get a clear “couldn't find” page with the search box ready; if the backend is not running you get an “API unreachable” message with a retry button. The app never shows a blank screen.

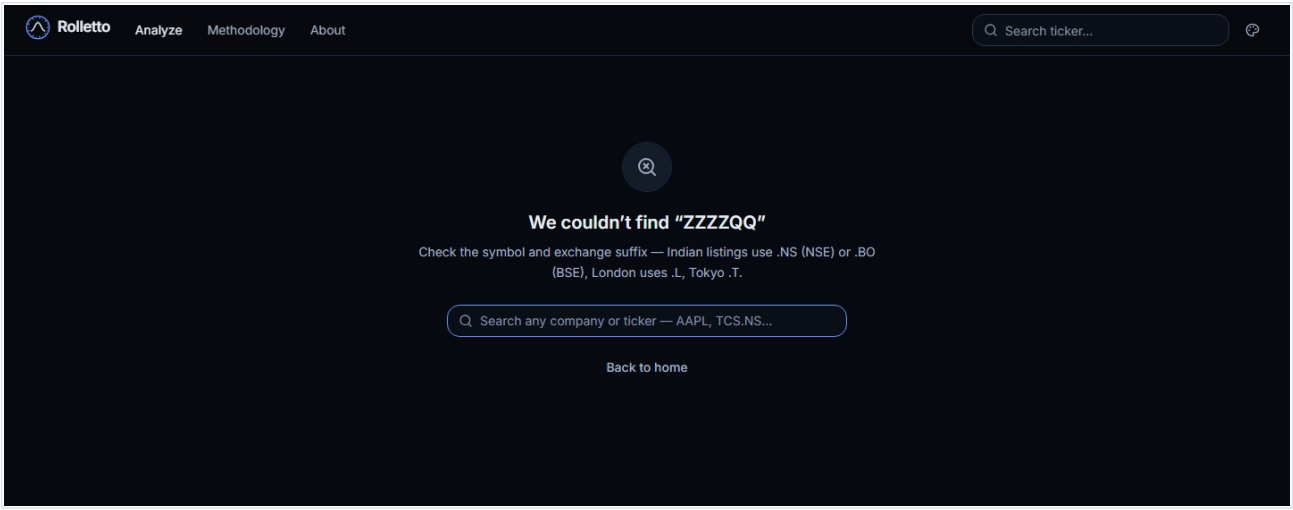


Figure A2 — An unknown ticker produces a clean error state, not a crash.

A4. The analysis workspace

Opening a company (for example `/analyze/KO`) shows the workspace. It has three areas:

- **Header** — company name, sector and exchange, price (with 1-year change and a sparkline), market cap, EV/EBITDA, a **data-quality badge** (click it to see every gap or fallback in the data) and the **LBO suitability score**. The Share, Excel and Memo PDF buttons are here too.
- **Assumptions panel** — every model input, grouped and collapsible (left by default; it can be moved right in Appearance).

- **Results** — seven tabs: Overview, DCF, LBO, Monte Carlo, Sensitivity, Comps & Football Field, Financials.

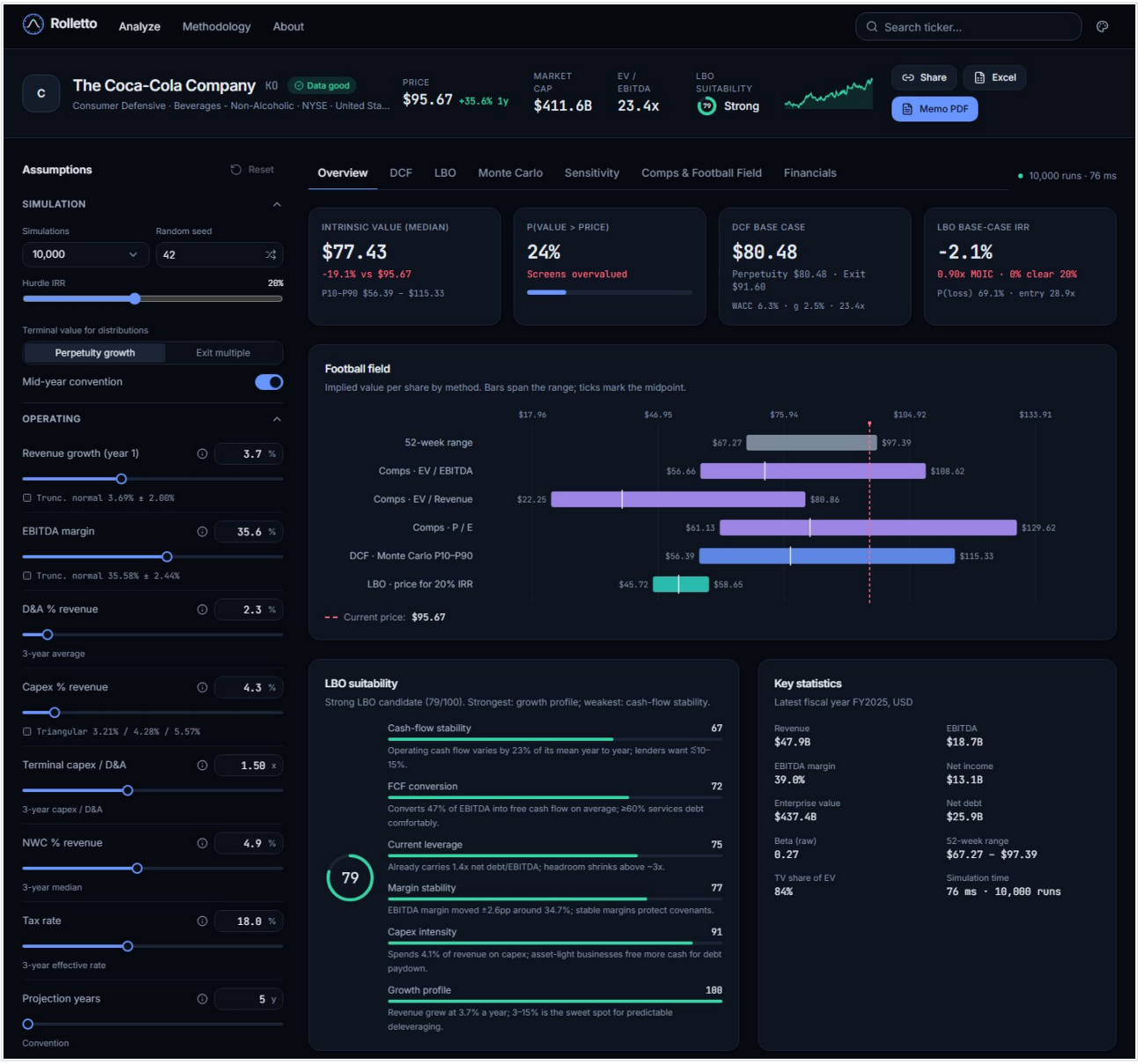


Figure A3 — The workspace for Coca-Cola (Overview tab).

The assumptions panel

Each assumption row shows its label, an tooltip, a numeric box and a slider. The tooltip explains the input, shows the automatically derived **default**, its **source** (for example "3-year average" or "Country default") and the **rationale** (for example "Average EBITDA margin 35.6%"). Change a value by dragging the slider or typing in the box and pressing Enter. A changed value is highlighted and gets a link that resets it to the default; **Reset** at the top resets everything.

The seven **stochastic** drivers (revenue growth, EBITDA margin, capex %, WACC, terminal growth, exit multiple and the senior interest rate) also show their probability distribution under the slider, for example "Trunc. normal 3.69% ± 2.00%". Click it to change the distribution type and its parameters. If you change the base value but not the distribution, the distribution is re-centred on your new value automatically.

Group	Inputs
Simulation	Number of simulations (1,000–100,000), random seed (same seed → identical results), hurdle IRR, which terminal value drives the distributions (perpetuity growth or exit multiple), mid-year discounting on/off.

Group	Inputs
Operating	Year-1 revenue growth, EBITDA margin, D&A % revenue, capex % revenue, terminal capex / D&A, NWC % revenue, tax rate, projection years (5–10).
Cost of capital	Risk-free rate, equity risk premium, beta, pre-tax cost of debt, target debt/capital, WACC, terminal growth, exit EV/EBITDA.
LBO entry	Offer premium, advisory fees % EV, financing fees % debt, minimum cash % revenue.
LBO debt	Senior debt / EBITDA, senior rate, senior amortisation, subordinated debt / EBITDA and rate, revolver size and rate, cash sweep.
LBO exit & covenants	Exit year (3–7), maximum net debt / EBITDA, minimum EBITDA / interest.

Note · WACC is computed from its components. If you change the risk-free rate, ERP, beta, cost of debt, tax rate or debt weight, WACC is recomputed; if you type a WACC directly, your number is used as-is.

Re-running and sharing

Every change triggers a new valuation 0.6 seconds after you stop adjusting (so dragging a slider does not fire hundreds of requests). While it runs, a progress bar and “Simulating 10,000 paths…” appear and the previous results stay visible, slightly dimmed. If a change is invalid (for example terminal growth above WACC), the last valid result stays on screen with a warning explaining why.

The page address updates as you work: `?a=…` encodes every changed assumption, distribution, setting and peer, and `#1bo` remembers the open tab. **Share** copies that address — anyone opening it sees exactly your analysis, and because the seed is stored the simulation results are identical.

A5. The result tabs

Overview

Four headline cards: the **median intrinsic value** from the simulation (with the P10–P90 range and upside vs price), **P(value > price)** with a verdict, the **DCF base case** under both terminal-value methods, and the **LBO base-case IRR** with MOIC and the probability of clearing the hurdle. Below are the football field, the LBO suitability breakdown and key statistics.

P(value > price)	Verdict shown	Meaning
≥ 65%	Screens undervalued	Most simulated values exceed today's price.
35% – 65%	Within fair-value range	The price is inside the central range of outcomes.
≤ 35%	Screens overvalued	Most simulated values are below today's price.

DCF

The full projection table (revenue → EBITDA → EBIT → taxes → NOPAT → + D&A − capex − ΔNWC = unlevered free cash flow → discount factor → present value), a revenue/EBITDA/margin chart joining the last three reported years to the forecast, the enterprise-to-equity bridge for both terminal-value methods, cross-checks (implied exit multiple, implied perpetual growth, terminal value as % of EV) and the simulated distribution of value per share with the current price marked.

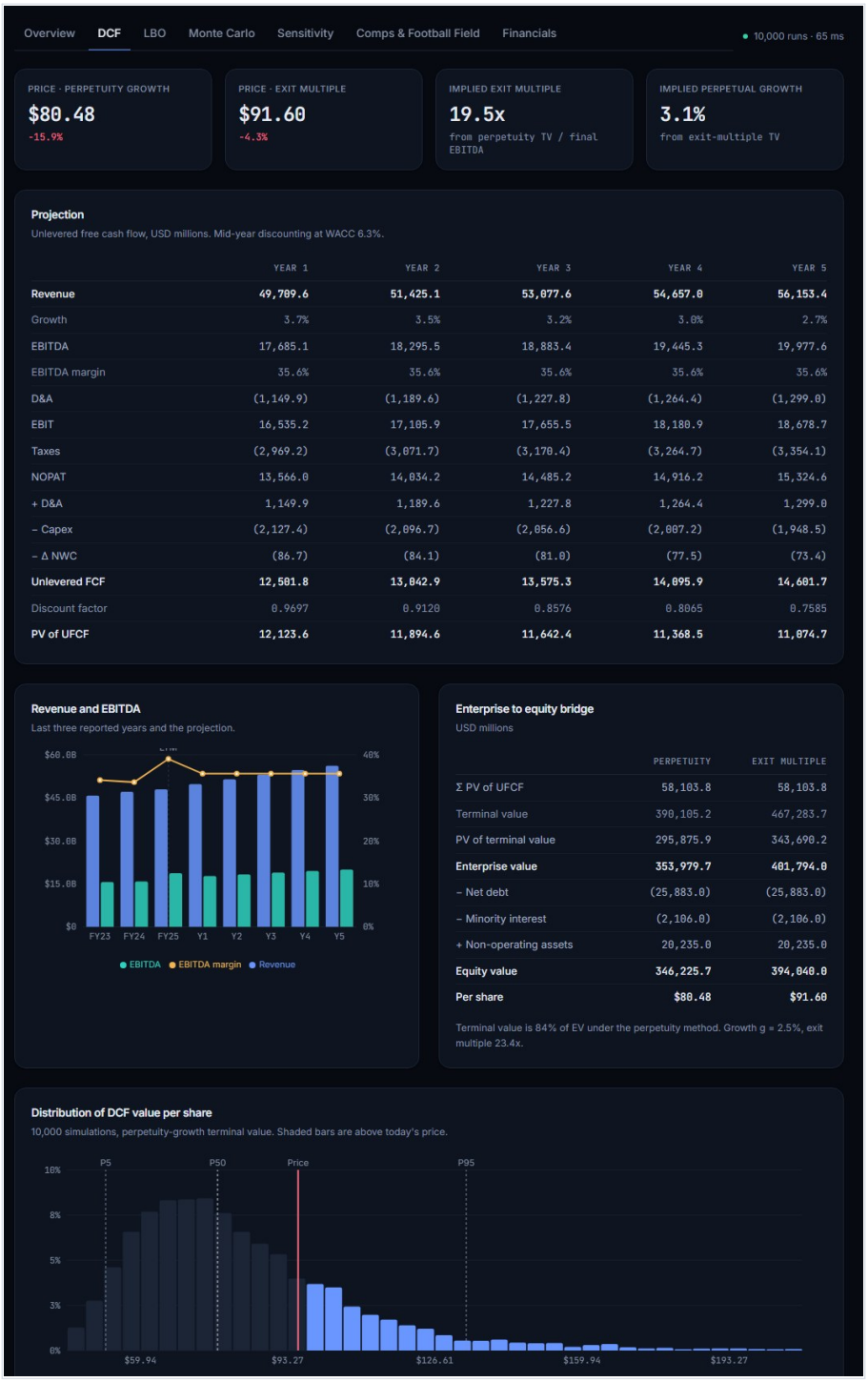


Figure A4 — DCF tab: projection, bridge and value distribution.

LBO

Sources and uses of funds, the year-by-year debt schedule (interest, taxes, levered free cash flow, mandatory amortisation, revolver, cash sweep, closing balances, leverage and coverage), base-case returns, the simulated IRR and MOIC distributions with the hurdle and 1.0x lines, and the probability of breaching each covenant by year. If the deal cannot be financed (for example EBITDA is negative), the tab explains why instead of showing numbers.

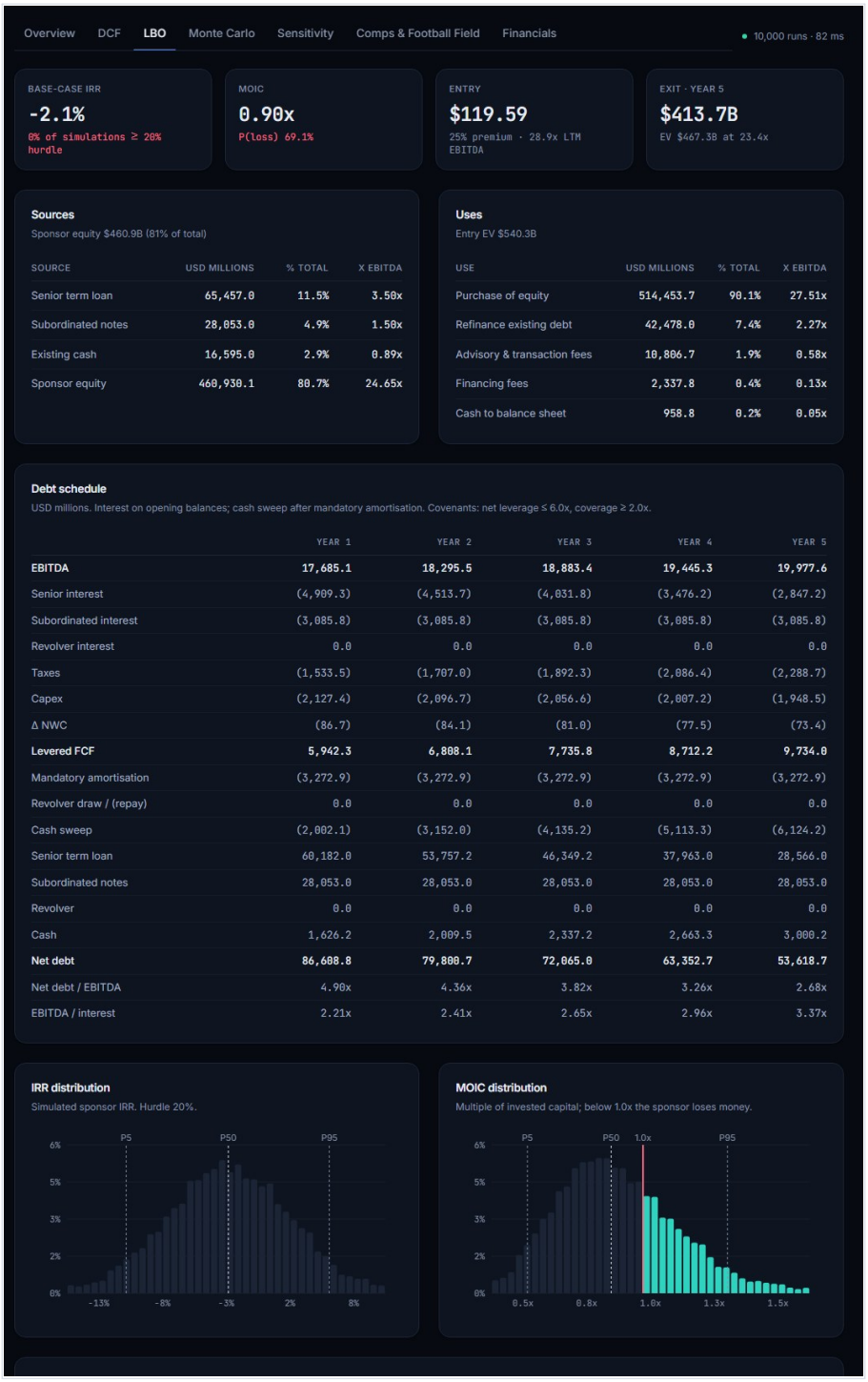


Figure A5 — LBO tab: sources & uses, debt schedule and return distributions.

Monte Carlo

The value-per-share histogram with P5/P50/P95 and price markers, a percentile table for share price, enterprise value, IRR and MOIC, the **convergence chart** (running mean with its 95% confidence band — a flat line means enough simulations), the enterprise value distribution, the distributions actually sampled for each driver, and the correlation matrix used.

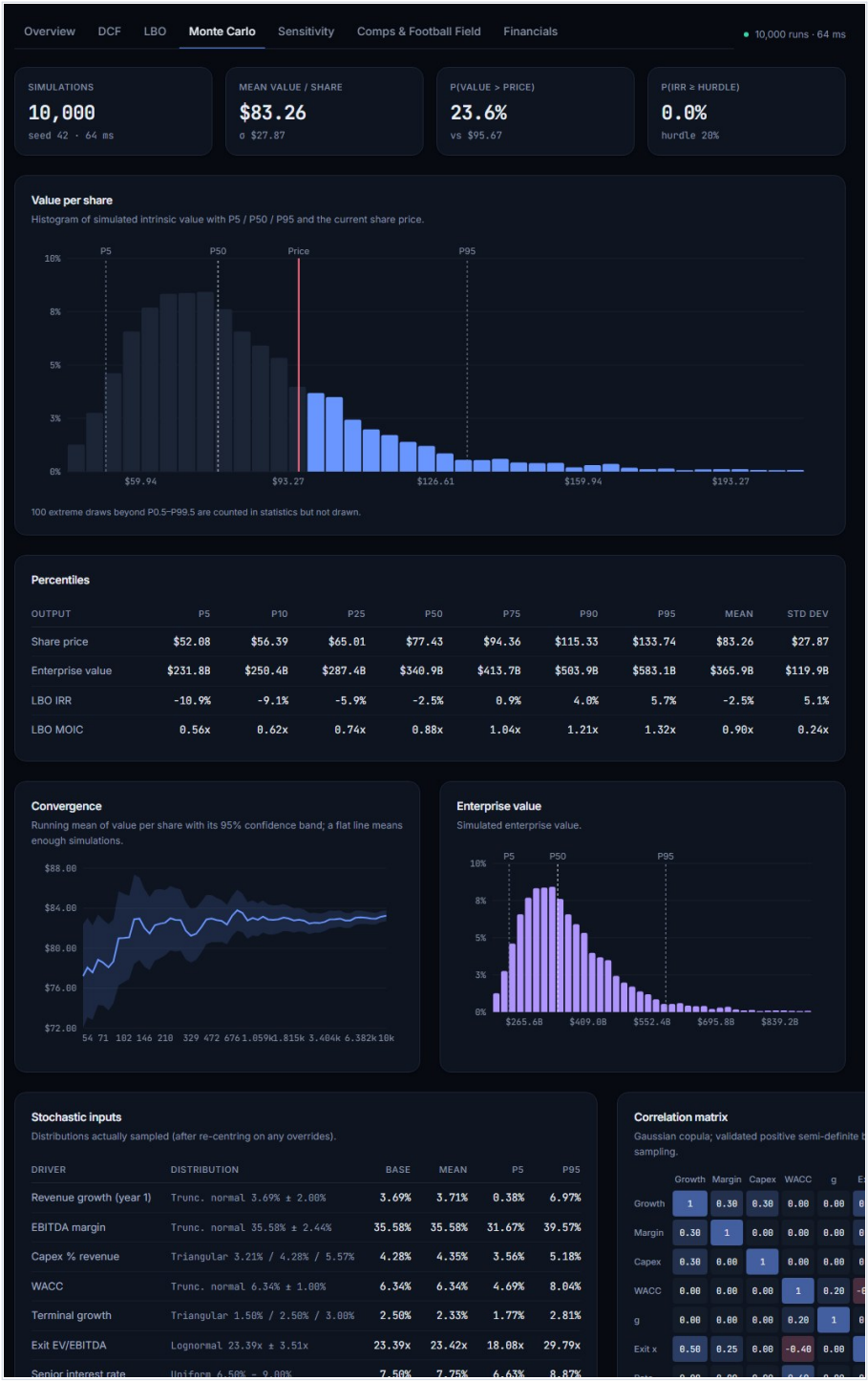


Figure A6 — Monte Carlo tab.

Sensitivity

Tornado charts for value per share and for sponsor IRR. Each bar swings one driver from its 10th to its 90th percentile while every other driver stays at its base value; bars are sorted by impact, and the labels show the input value at each end. Red is the side below the base case, green above. A driver with no bar does not affect that output (for example the exit multiple does not affect the perpetuity-growth DCF).

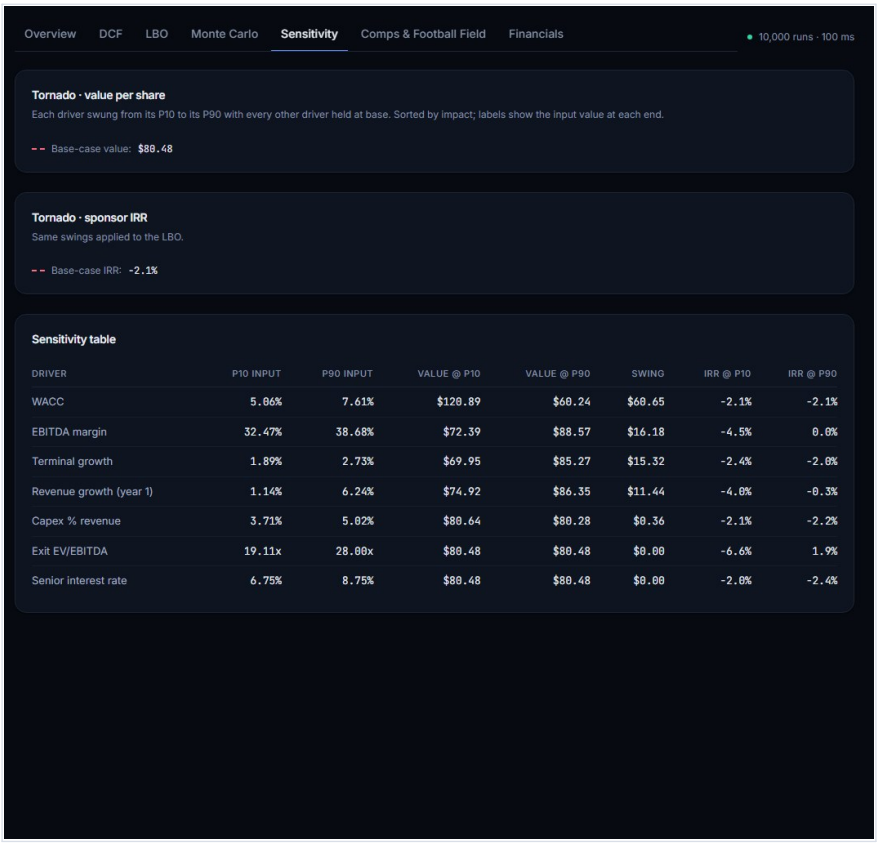


Figure A7 — Sensitivity tab.

Comps & Football Field

The football field puts every method on one axis: the 52-week trading range, the share price implied by peer EV/EBITDA, EV/Revenue and P/E (interquartile ranges), the DCF Monte Carlo P10–P90 and the LBO range (the maximum price a sponsor could pay and still earn the hurdle IRR). The peer table lists each peer's multiples; remove peers with □ or add any ticker. “Automatic peers” restores the default set. Peers with no meaningful multiples (losses, missing data) are shown but excluded.

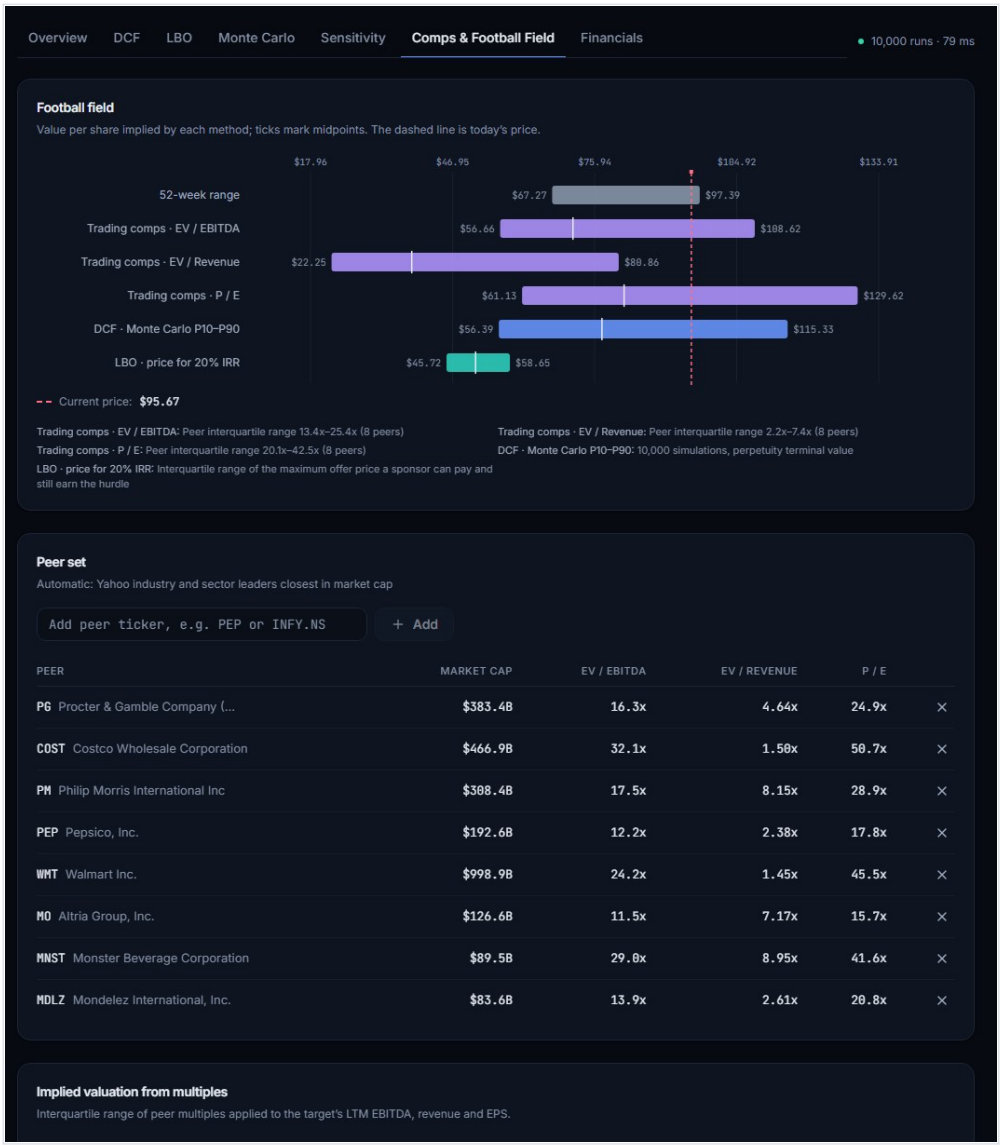


Figure A8 — Comps & Football Field tab.

Financials

The cleaned historical income statement, balance sheet and cash flow (up to five years) and key ratios. Missing values are shown as "—" and are never estimated. Figures are in millions — or crores (₹ Cr) for Indian companies when the Lakh/crore switch is on.

A6. Financial companies

Banks, insurers, NBFCs, broker-dealers and REITs are detected automatically from their industry. For these businesses debt is raw material rather than financing, so "unlevered free cash flow" and "EBITDA" are not meaningful and a DCF or LBO would be wrong. Rolletto shows an explanation and two equity-side valuations instead: a justified price-to-book from return on equity, and a dividend discount model (Part B, section B8).

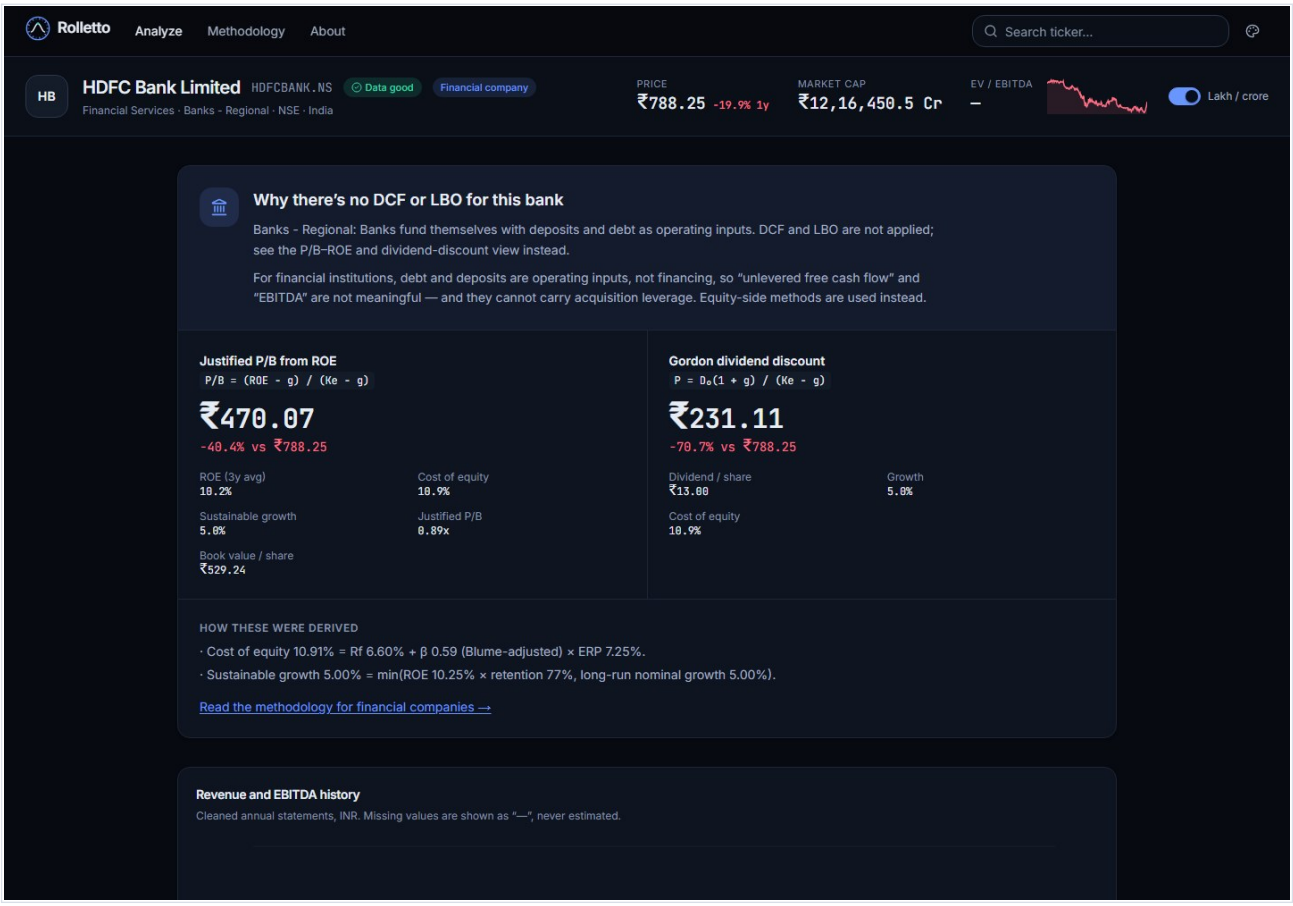


Figure A9 — HDFC Bank: the financial-company view, with Indian lakh/crore formatting.

A7. Exports, appearance and formats

Excel workbook

Excel downloads a formatted workbook with seven tabs: Summary, Assumptions, DCF, LBO, Monte Carlo, Comps and Historicals. It follows banking conventions — **blue** cells are inputs, black cells are formulas, grey italic cells are values from the engine. The DCF tab is built entirely from live formulas that reference the Assumptions tab, so you can change a blue input in Excel and the valuation recalculates. A check row compares the Excel result with the engine's (it reconciles to zero).

Memo PDF

Memo PDF produces a one-page memo: company overview, valuation summary table, football field, automatically generated key risks (terminal-value dependence, most sensitive drivers, outcome spread, covenant risk, data warnings) and a plain-language framing of the result. It is labelled as educational and not investment advice.

Appearance

The palette icon in the navigation bar opens the Appearance panel. Choose one of six themes — Midnight, Terminal, Graphite, Ivory, Aurora or Monaco — each in light and dark, then adjust light/dark/system mode, interface typeface, number typeface, corner style, density and which side the assumptions panel sits on. Choices are saved in your browser and applied before the page paints.

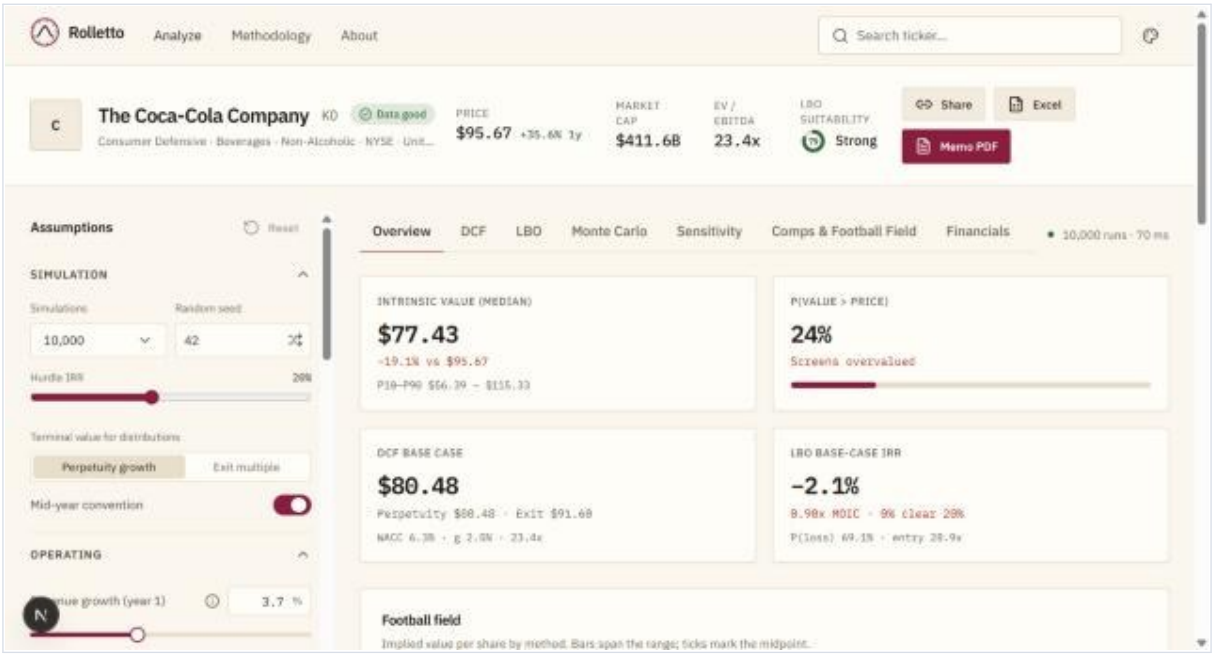


Figure A10 — Ivory theme (light): editorial palette, IBM Plex, sharp corners.

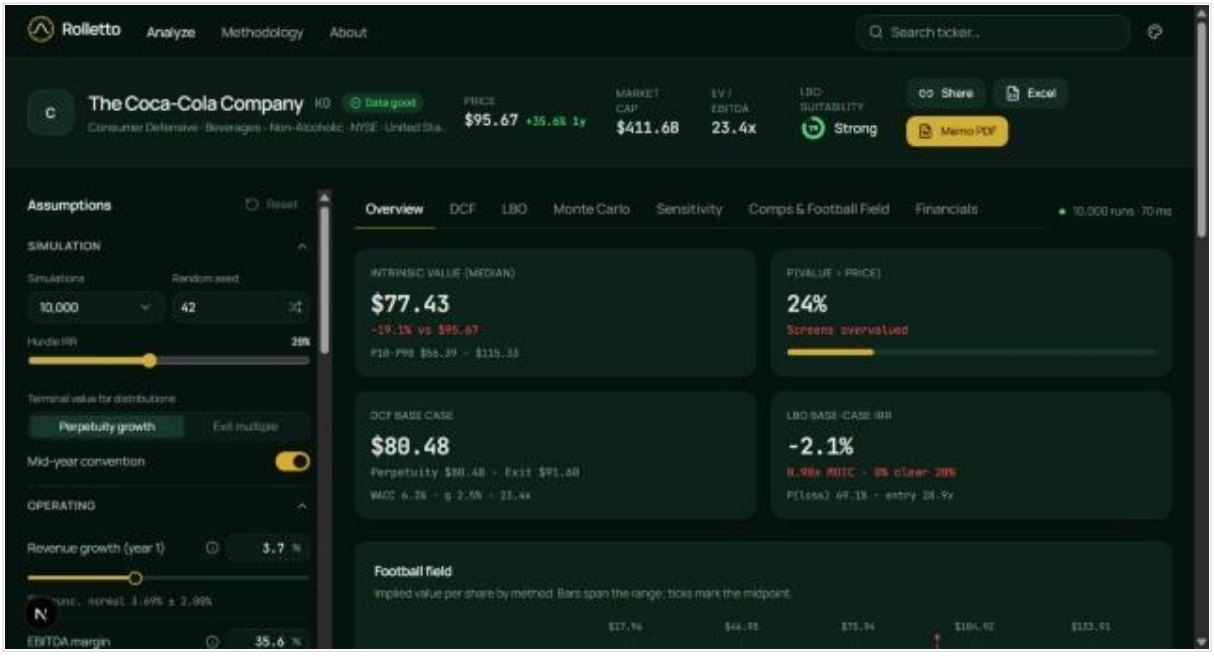


Figure A11 — Monaco theme (dark): casino-felt green with gold accents.

Number formats

Values are shown in the company's reporting currency with its symbol (₹, $, €, £, ¥ …). For Indian rupees a Lakh/crore switch in the header formats amounts as 1,23,456 and ₹ Cr. When a listing trades in a different currency from its statements (for example an ADR), the price is converted at the live exchange rate and a data-quality note says so.

A8. Reading the results well

- **Look at the range, not the point.** A median value 10% above the price with a P10–P90 range of −40% to +80% is a weak signal.
- **Check terminal value share.** If terminal value is more than ~75% of EV (shown on Overview and DCF), the result depends mostly on long-run assumptions — test terminal growth and the exit multiple first.
- **Use the tornado to decide what to research.** The top two bars are where better information has the most value.
- **Compare methods.** If the DCF range and the comps ranges do not overlap, find out why before trusting either.
- **Read the data-quality badge.** Fallbacks (for example EBITDA derived as EBIT + D&A) and anomalies are listed there.
- **An LBO IRR that looks poor is normal for expensive, large companies.** A 25% premium on a richly valued stock with modest leverage rarely clears 20%; the LBO view is most informative for mid-sized, cash-generative businesses.

Part B — Valuation methods

This part documents every calculation exactly as the code performs it. Formulas are given in the notation used throughout: t is the forecast year (1 … T), g is growth, m is EBITDA margin and WACC is the discount rate.

B1. Data and cleaning

All market data enters through a **DataProvider** interface; the current implementation uses Yahoo Finance via the open-source *yfinance* library. Requests are cached in memory (12 hours for financial statements, 15 minutes for prices), retried up to three times with exponential backoff, and cut off after 12 seconds. A Yahoo "404" is treated as a definitive "ticker not found" and is not retried.

Normalising line items

Different companies label the same item differently. For each standard field the cleaner tries a list of labels in order and takes the first one that has a finite value:

Field	Labels tried (in order)	Fallback when missing
Revenue	Total Revenue → Operating Revenue → Revenue	Period is dropped (never invented)
EBITDA	EBITDA → Normalized EBITDA	EBIT + D&A (recorded as a fallback)
EBIT	EBIT → Operating Income → Total Operating Income As Reported	EBITDA − D&A
D&A	Depreciation And Amortization (cash flow) → Reconciled Depreciation	—
Total debt	Total Debt	Long-term + current debt
Cash	Cash, Cash Equivalents & Short-Term Investments → Cash And Cash Equivalents	—
Operating NWC	Receivables + Inventory − Payables	(Current assets − cash) − (current liabilities − current debt)
Capex	Capital Expenditure → Purchase of PPE	— (stored as a positive outflow)

Data quality

Every company gets a grade (good / fair / poor) and a list of findings, shown in the data-quality badge. Errors make the company unusable for DCF/LBO (Rolletto refuses rather than computing on bad data); warnings and info items are shown but do not block.

Severity	Checks
Error	No statements; latest revenue zero or negative; EBITDA unavailable.
Warning	Fewer than 3 years; gap in reported years; revenue change over 100% in a year; negative equity; negative EBITDA; capex, D&A or debt not reported.
Info	Each derived figure; dropped periods; missing beta; currency conversion for listings that trade in a different currency.

B2. Default assumptions

Defaults come from the company's own history, bounded so that one unusual year cannot drive a perpetuity. Each default is returned with its source and rationale and every one can be overridden.

Assumption	Default rule	Bounds
Revenue growth (yr 1)	Compound annual growth over the available history	−5% to 25%
EBITDA margin	3-year average of EBITDA / revenue	−10% to 60%

Assumption	Default rule	Bounds
D&A % revenue	3-year average (3% placeholder if missing, flagged)	—
Capex % revenue	3-year average (set equal to D&A if missing)	—
Terminal capex / D&A	3-year average capex / D&A	1.0x to 1.5x
NWC % revenue	3-year median of operating NWC / revenue (0% if missing)	−20% to 40%
Tax rate	3-year effective rate if 5–40%, else the country statutory rate	cap 35%
Risk-free, ERP	Country table (configurable), e.g. US 4.25% / 4.5%, India 6.6% / 7.25%	—
Beta	Provider beta, Blume-adjusted; 1.0 if missing	0.4 to 2.5
Cost of debt	Latest interest expense / average total debt	Rf + 1% to 15%
Debt weight	Market D / (D + E)	0% to 60%
Terminal growth	Country long-run nominal growth (US 2.5%, India 5%)	≤ WACC − 2%
Exit EV/EBITDA	Current EV / latest EBITDA	4x to 25x
Offer premium, fees	25% premium; 2% advisory; 2.5% financing; min cash 2% of revenue	—
Debt package	Total ≤ min(5.0x, 60% of entry EV/EBITDA); senior ≤ 3.5x (70%), rest subordinated	—
Rates	Senior = Rf + country spread; sub = senior + 3.5%; revolver = senior − 0.25%	—
Other LBO	5% amortisation, 75% sweep, 0.5x revolver, 5-year hold, covenants 6.0x / 2.0x	—

B3. Cost of capital

The cost of equity uses CAPM with a Blume-adjusted beta, which pulls the raw regression beta toward the market average of 1 because betas mean-revert:

$$\beta_{adj} = 0.67 \times \beta_{raw} + 0.33$$

$$K_e = R_f + \beta_{adj} \times ERP$$

$$WACC = (E/V) \times K_e + (D/V) \times K_d \times (1 - t)$$

For KO today: raw beta 0.27 → adjusted 0.51; K_e = 4.25% + 0.51 × 4.50%; with K_d 5.25%, tax 18.0% and D/V 9.4%, WACC = **6.34%**.

B4. Discounted cash flow

Projection

Revenue growth fades in a straight line from the year-1 rate toward terminal growth, so the last forecast year is close to steady state:

$$g_t = g_1 + (g_\infty - g_1) \times (t - 1) / T$$

$$Revenue_t = Revenue_{t-1} \times (1 + g_t)$$

Margins, D&A and working capital are held as constant shares of revenue. Capex fades from its current level to a steady-state level of k × D&A by the final year (k = terminal capex / D&A):

$$capex\%_t = c_1 + (k \times d - c_1) \times (t - 1) / (T - 1)$$

$$EBIT_t = Revenue_t \times (m - d)$$

$$Taxes_t = max(0, EBIT_t) \times tax$$

$$\Delta NWC_t = nwc\% \times (Revenue_t - Revenue_{t-1})$$

$$UFCF_t = EBIT_t - Taxes_t + D\&A_t - Capex_t - \Delta NWC_t$$

Why · Why fade capex? Many companies are mid-way through an investment cycle (Reliance, Microsoft). Holding today's capex level forever would make the terminal year — which drives most of the value — a peak-investment year, which can even produce a negative value. Fading to a multiple of D&A makes the terminal year a steady state, as Rosenbaum & Pearl recommend.

Terminal value — both methods

$$TV^{PGM} = UFCF_T \times (1 + g_\infty) / (WACC - g_\infty)$$

$$TV^{EMM} = exit\ multiple \times EBITDA_T$$

Discounting and the bridge to price

$$EV = \Sigma_t\ UFCF_t / (1 + WACC)^{t - ½} + TV / (1 + WACC)^{T - ½\ (PGM)\ or\ T\ (EMM)}$$

$$Equity\ value = EV - net\ debt - minority\ interest + non\text{-}operating\ assets$$

$$Price = Equity / shares$$

Convention · With mid-year discounting on, cash flows are discounted at t − ½ because they arrive through the year. The perpetuity-growth terminal value is discounted at T − ½ because its cash flows also arrive mid-year; the exit-multiple value is a sale price at the end of year T, so it is discounted at T. With mid-year off, everything uses whole years.

Cross-checks reported with every DCF: the EV/EBITDA multiple implied by the perpetuity value, the perpetual growth implied by the exit-multiple value (solving TV = UFCF(1+g)/(WACC−g) for g), and terminal value as a share of EV.

Live example — Coca-Cola DCF ($ millions)

	Year 1	Year 2	Year 3	Year 4	Year 5
Revenue	49,710	51,425	53,078	54,657	56,153
Growth	3.7%	3.5%	3.2%	3.0%	2.7%
EBITDA	17,685	18,295	18,883	19,445	19,978
EBIT	16,535	17,106	17,656	18,181	18,679
Taxes	2,969	3,072	3,170	3,265	3,354
Capex	2,127	2,097	2,057	2,007	1,948
ΔNWC	87	84	81	77	73
Unlevered FCF	12,502	13,043	13,575	14,096	14,602
Discount factor	0.9697	0.9120	0.8576	0.8065	0.7585
PV of UFCF	12,124	11,895	11,642	11,369	11,075

Bridge	Perpetuity growth	Exit multiple
Σ PV of UFCF	$58.1B	$58.1B
Terminal value	$390.1B	$467.3B
PV of terminal value	$295.9B	$343.7B
Enterprise value	$354.0B	$401.8B
− Net debt − minorities + non-operating assets	$-7.8B	$-7.8B
Equity value	$346.2B	$394.0B
Value per share (price $95.67)	$80.48	$91.60

Assumptions: year-1 growth 3.7%, EBITDA margin 35.6%, WACC 6.34%, terminal growth 2.5%, exit multiple 23.4x, mid-year convention on. Terminal value is 84% of EV under the perpetuity method.

B5. Leveraged buyout

Entry — sources and uses

$$\textit{Offer price} = \textit{price} \times (1 + \textit{premium})$$

$$\textit{Entry EV} = \textit{offer equity} + \textit{existing debt} - \textit{existing cash}$$

$$\textit{Uses} = \textit{equity purchase} + \textit{refinanced debt} + \textit{advisory fees (\% EV)} + \textit{financing fees (\% new debt)} + \textit{minimum cash}$$

$$\textit{Sources} = \textit{senior loan} + \textit{subordinated notes} + \textit{existing cash} + \textbf{\textit{sponsor equity (the balancing item)}}$$

Debt is sized as multiples of last-twelve-months EBITDA. If EBITDA is not positive, or the debt plus existing cash would exceed the purchase price, the LBO is marked not feasible with the reason.

The annual debt schedule

$$\textit{Interest}_t = \textit{senior}_{t-1} \times r_s + \textit{sub}_{t-1} \times r_{sub} + \textit{revolver}_{t-1} \times r_{rev}$$

$$\textit{Levered FCF}_t = \textit{EBITDA}_t - \textit{Interest}_t - \max(0, \textit{EBIT}_t - \textit{Interest}_t) \times \textit{tax} - \textit{Capex}_t - \Delta\textit{NWC}_t$$

Cash is applied each year in this order:

- Mandatory amortisation of the senior loan (a fixed % of the original principal).
- If cash would fall below the minimum, the revolver is drawn (up to its commitment).
- Otherwise any revolver balance is repaid first, then the **cash sweep** pays down the senior loan with the chosen % of the excess.
- The subordinated note is a bullet: it is repaid only at exit.

Convention · Interest is charged on opening balances. Average-balance interest is slightly more precise but creates a circular reference (interest depends on repayment, which depends on interest); opening balances keep the model exact and fast.

Exit, returns and covenants

$$\text{Exit EV} = \text{exit multiple} \times \text{EBITDA}_N$$

$$\text{Exit equity} = \max(\text{Exit EV} - \text{net debt}_N, 0)$$

$$\text{MOIC} = \text{Exit equity} / \text{Sponsor equity}$$

$$\text{IRR} = \text{MOIC}^{1/N} - 1$$

The IRR formula is exact because the sponsor puts money in once and takes it out once (no dividends in between). Each year is tested against a maximum net debt / EBITDA and a minimum EBITDA / interest covenant; a negative or zero EBITDA always counts as a breach.

The LBO bar on the football field

Holding the debt package fixed, sponsor equity is a straight-line function of the offer price, so the highest price that still earns the hurdle IRR can be solved exactly for every simulation:

$$\text{Required equity} = \text{Exit equity} / (1 + \text{hurdle})^N$$

$$\text{Offer equity} = (\text{Required equity} - \text{fixed costs}) / (1 + \text{advisory \%})$$

The football field shows the interquartile range of that price across all simulations.

Live example — Coca-Cola LBO (base case)

Sources	$ bn	x EBITDA
Senior term loan	65.5	3.50x
Subordinated notes	28.1	1.50x
Existing cash	16.6	0.89x
Sponsor equity	460.9	24.65x

Entry at $119.59 per share (25% premium) is 28.9x LTM EBITDA. Base-case IRR -2.1%, MOIC 0.90x; across 10,000 simulations P(IRR ≥ 20%) = 0.0% and P(loss) = 69.1%. As expected for a richly valued mega-cap: the entry multiple is far above what the exit multiple and modest leverage can support.

B6. Monte Carlo simulation

Stochastic drivers and distributions

Distribution	Parameters	Used by default for
Normal	mean, σ	—
Truncated normal	mean, σ, low, high	Revenue growth (σ = historical volatility, 2–8%), EBITDA margin, WACC (σ = 1%)
Triangular	low, mode, high	Capex % (−25% / +30% of base), terminal growth (−1% / +0.5%)
Uniform	low, high	Senior interest rate (−1% / +1.5%)
Lognormal	mean, σ of the variable	Exit multiple (σ = 15% of the mean; always positive)

The subordinated and revolver rates move in parallel with the senior rate. Inside a simulation, terminal growth is capped at WACC − 0.5% (the share of capped draws is reported), the exit multiple is floored at 0.5x and capex and rates at zero.

Correlated sampling — the Gaussian copula

Drivers are not independent: fast growers get higher multiples; higher discount rates come with higher borrowing costs and lower multiples. Rolletto samples them jointly with a Gaussian copula:

$$1.\ \text{Draw } Z \sim N(0, I),\ \text{an } n \times k \text{ matrix of independent standard normals}$$

$$2.\ \text{Correlate: } Z' = Z\,L^{\intercal},\ \text{where } \Sigma = L\,L^{\intercal} \text{ is the Cholesky factor of the correlation matrix}$$

$$3.\ \text{Map to uniforms: } U = \Phi(Z)$$

$$4.\ \text{Apply each driver's inverse CDF: } X_j = F_j^{-1}(U_j)$$

Each driver keeps exactly its chosen distribution while their rank correlation follows Σ. The matrix is checked to be positive semi-definite first (a set of correlations that cannot coexist is rejected, with the nearest valid matrix suggested).

Default correlation	ρ	Reason
Revenue growth ↔ exit multiple	+0.50	Markets pay up for growth
WACC ↔ senior rate	+0.60	Shared interest-rate environment
WACC ↔ exit multiple	−0.40	Higher discount rates compress multiples
Revenue growth ↔ EBITDA margin	+0.30	Operating leverage
Revenue growth ↔ capex %	+0.30	Growth requires investment
EBITDA margin ↔ exit multiple	+0.25	Quality earns a premium
WACC ↔ terminal growth	+0.20	Inflation lifts both

Outputs

Every output is a distribution summarised by mean, standard deviation, percentiles (P5 … P95) and a 40-bin histogram over the central 99% of draws. Probabilities are simple frequencies across simulations. Convergence is the running mean with a 95% band:

$$\bar{x}_n \pm 1.96 \times s_n / \sqrt{n}$$

KO, 10,000 simulations	P5	P25	P50	P75	P95	Mean
Value per share	$52.08	$65.01	$77.43	$94.36	$133.74	$83.26
LBO IRR	-10.9%	-5.9%	-2.5%	0.9%	5.7%	-2.5%

P(value > price $95.67) = **23.6%**. The whole simulation took 49 ms.

Sensitivity (tornado)

For each driver Rolletto evaluates the model twice — at the driver's 10th and 90th percentiles with everything else at base — all in a single batched run of the same engine. Bars are sorted by the size of the swing.

Driver	P10 input	P90 input	Value @ P10	Value @ P90	Swing
WACC	5.06%	7.61%	$120.89	$60.24	$60.65
EBITDA margin	32.47%	38.68%	$72.39	$88.57	$16.18
Terminal growth	1.89%	2.73%	$69.95	$85.27	$15.32
Revenue growth (year 1)	1.14%	6.24%	$74.92	$86.35	$11.44
Capex % revenue	3.71%	5.02%	$80.64	$80.28	$0.36

B7. Comparable companies and the football field

Peers: for Indian listings, a curated list of large peers in the same industry; otherwise Yahoo's industry and sector leaders, of which the eight closest in market capitalisation (on a log scale) are kept. You can replace the set entirely. Multiples outside sensible bounds are treated as not meaningful (EV/EBITDA 0.5–60x, EV/Revenue 0.05–40x, P/E 1–150x).

$$\text{Implied price (EV multiples)} = (\text{multiple} \times \text{target metric} - \text{net debt} - \text{minorities} + \text{non-operating assets}) / \text{shares}$$

$$\text{Implied price (P/E)} = P/E \times EPS$$

Each method uses the interquartile range (P25–P75) of peer multiples. A negative implied price (possible for low EV/Revenue peers and an indebted target) is shown as not meaningful rather than plotted.

B8. LBO suitability score

A 0–100 score from six sub-scores. Each sub-score maps one historical metric linearly between a "bad" and a "good" threshold (clamped to 0–100). If a metric cannot be computed, its weight is redistributed across the others.

Sub-score	Weight	Metric	0 at	100 at
Cash-flow stability	20%	Coefficient of variation of operating cash flow	50%	10%
FCF conversion	20%	Average (OCF − capex) / EBITDA	15%	60%
Current leverage	15%	Net debt / EBITDA	4.0x	0.5x
Margin stability	15%	Std. deviation of EBITDA margin	8pp	1pp
Capex intensity	15%	Capex / revenue	15%	3%
Growth profile	15%	Revenue CAGR (100 between 3% and 15%)	−5% / 40%	3–15%

Grades: Strong ≥ 75, Moderate ≥ 55, Weak ≥ 35, otherwise Poor.

B9. Financial companies

Detected from the industry (banks, insurance, credit services / NBFCs, mortgage finance, capital markets, REITs). Asset managers and exchanges are fee businesses and stay on the normal path.

$$\text{Justified } P/B = (ROE - g) / (K_e - g)$$

$$\text{Value} = \text{book value per share} \times \text{justified } P/B$$

$$\text{Dividend discount: Value} = D_0 \times (1 + g) / (K_e - g)$$

$$g = \min(\, ROE \times (1 - \text{payout}),\ \text{long-run nominal growth},\ K_e - 1\% \,)$$

ROE is the average of net income / average equity over the last three years (the provider's trailing ROE if history is insufficient).

B10. Hand-checked example — Acme Corp

The test suite includes a fictional company whose numbers are simple enough to verify with a calculator. The engine must reproduce these figures to the cent. Inputs: revenue 1,000; growth fading 8% → 4%; EBITDA margin 20%; D&A 4%; capex 5%; NWC 10% of revenue; tax 25%; WACC 10%; terminal growth 3%; exit 8.0x; net debt 200; 100 shares at 12.00.

	Year 1	Year 2	Year 3	Year 4	Year 5
Revenue	1,080.00	1,155.60	1,224.94	1,286.18	1,337.63
EBITDA	216.00	231.12	244.99	257.24	267.53
NOPAT	129.60	138.67	146.99	154.34	160.52
+ D&A − capex − ΔNWC	−18.80	−19.12	−19.18	−18.99	−18.52
Unlevered FCF	110.80	119.56	127.81	135.36	141.99

Result (end-of-year discounting)	Perpetuity growth	Exit multiple
Σ PV of UFCF	476.18	476.18
Terminal value	2,089.35	2,140.21
Enterprise value	1,773.50	1,805.08
Value per share	15.73	16.05
With mid-year convention	16.60	16.28

LBO: a 25% premium gives an offer of 15.00 and entry EV of 1,700 (8.5x). Uses 1,872 = 1,500 equity + 300 refinancing + 34 advisory + 18 financing + 20 minimum cash; sources = 600 senior (3.0x) + 300 subordinated (1.5x) + 100 existing cash + **872 sponsor equity**. Year 1: interest 48 + 33 = 81; taxes (172.80 − 81) × 25% = 22.95; levered FCF = 216 − 81 − 22.95 − 54 − 8 = **50.05**; mandatory amortisation 30 and a 20.05 sweep leave the senior loan at 549.95. After five years the senior loan is 231.10, exit equity is 2,140.21 − 511.10 = 1,629.11, MOIC = 1.8682x and **IRR = 13.31%**.

Verification · The same figures were cross-checked with an independent script that does not use the engine, and the Excel export's live formulas were recalculated in LibreOffice and reconcile with the engine exactly.

Part C — How the code works

C1. Architecture

Rolletto is a monorepo with a Python backend and a TypeScript frontend that talk over a JSON API. All finance lives in the backend; the frontend only displays results and collects inputs.

Layer	Technology	Responsibility
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4, Radix UI, Recharts, TanStack Query, Zod	Pages, workspace UI, charts, input state, share links, appearance themes
API	FastAPI, Pydantic v2, slowapi	Routes, validation, CORS, rate limits, consistent errors, OpenAPI docs
Service	Python	Orchestrates data loading, cleaning, classification and the engines
Data	yfinance behind a DataProvider protocol, cachetools, tenacity	Fetching, caching, retries, normalisation, data quality
Engine	NumPy, SciPy	Assumptions, WACC, DCF, LBO, Monte Carlo, sensitivity, comps, suitability, financial-company valuation
Exports	openpyxl, ReportLab	Excel workbook with live formulas; one-page PDF memo

Folder map

Path	Contents
`backend/app/api/`	routes.py (all endpoints), deps.py (service wiring and rate limiter)
`backend/app/core/`	config.py (settings, country defaults), cache.py, errors.py, logging.py, currency.py
`backend/app/data/`	provider.py (the interface), yfinance_provider.py, normalize.py, quality.py, peers.py
`backend/app/engine/`	assumptions.py, wacc.py, dcf.py, lbo.py, distributions.py, montecarlo.py, valuation.py (glue), suitability.py, classify.py, financials_valuation.py, comps.py
`backend/app/export/`	excel.py, memo.py
`backend/app/schemas/`	Pydantic models — the API contract
`backend/app/services/`	company.py — CompanyService.company() and .valuation()
`backend/tests/`	fixtures/acme.py and test_*.py (65 tests)
`frontend/src/app/`	Routes: page.tsx (home), analyze/, methodology/, about/, 404, error, sitemap, robots, OG image, themes.css
`frontend/src/components/`	ui/ (primitives), layout/, analyze/ (workspace, panel, tabs), charts/, home/, methodology/
`frontend/src/lib/`	http.ts, api.ts, schemas.ts (Zod), format.ts, share.ts, appearance.ts

C2. The life of a valuation request

- **Browser.** You move a slider. The workspace stores the override, waits 0.6 s for you to stop, then TanStack Query sends `POST /api/valuation` with the ticker, overrides, distributions, correlation, settings and peers.
- **Route.** FastAPI validates the body against `ValuationRequest`, applies the rate limit and calls `CompanyService.valuation()`.
- **Load.** The service fetches profile, market data, statements and price history through the provider (cached), normalises the statements, classifies the company and grades data quality. Financial companies and unusable data stop here with a clear error.
- **Assumptions.** `derive_defaults()` builds every default; `resolve()` applies your overrides, recomputes WACC if needed and re-centres distributions.

- **Engines.** `evaluate()` runs the DCF and LBO for the base case (a batch of 1); `simulate()` samples 10,000 correlated driver sets and runs the same function on all of them at once; `tornado()` runs 15 one-at-a-time cases in one batch; comps and the football field are assembled.
- **Response.** Everything is returned as one JSON document. The frontend validates it with Zod and renders the tabs.

C3. Backend — data layer

The provider interface is the seam that keeps yfinance replaceable. Any class with these methods can be plugged in:

backend/app/data/provider.py

```
class DataProvider(Protocol):
    name: str

    def search(self, query: str) -> list[SearchResult]:
        """Return up to ~8 equity matches for a free-text query or partial ticker."""
        ...

    def get_profile(self, ticker: str) -> CompanyProfile:
        """Return company metadata. Raises ``TickerNotFoundError`` for unknown tickers."""
        ...

    def get_financials(self, ticker: str) -> tuple[Financials, NormalizationReport]:
        """Return up to 5 years of normalised annual statements plus notes on any fallbacks used."""
        ...

    def get_price_history(self, ticker: str, period: str = "1y") -> list[PricePoint]:
        """Return daily closes for ``period`` (e.g. '1y'), in the trading currency."""
        ...

    def get_market_data(self, ticker: str) -> MarketData:
        """Return price, shares, market cap, beta, debt and cash, converted to the reporting currency."""
        ...

    def get_peer_candidates(self, ticker: str, profile: CompanyProfile) -> list[str]:
        """Return candidate peer tickers in the same industry, most relevant first."""
        ...

    def get_peer_snapshot(self, ticker: str) -> PeerSnapshot | None:
```

Normalisation picks the first label that has a finite value, so a company that reports “Operating Revenue” instead of “Total Revenue” is handled transparently:

backend/app/data/normalize.py

```
def _pick(frame: pd.DataFrame, column: pd.Timestamp, labels: tuple[str, ...]) -> float | None:
    if frame.empty or column not in frame.columns:
        return None
    for label in labels:
        if label in frame.index:
            value = _finite(frame.at[label, column])
            if value is not None:
                return value
    return None
```

Every yfinance call goes through one wrapper that adds a hard timeout, exponential-backoff retries and error translation. A 404 is recognised inside the retry loop so that unknown tickers fail fast:

backend/app/data/yfinance_provider.py

```
    def _call(self, label: str, fn: Callable[[], T]) -> T:
        """Run a yfinance call with a hard timeout and exponential-backoff retries."""

        @retry(
            stop=stop_after_attempt(self._settings.provider_retries),
            wait=wait_exponential(multiplier=0.5, max=4),
            retry=retry_if_not_exception_type(TickerNotFoundError),
            reraise=True,
        )
        def attempt() -> T:
```

backend/app/data/yfinance_provider.py (continued)

```
            future = self._pool.submit(fn)
            try:
                return future.result(timeout=self._settings.provider_timeout_s)
            except FutureTimeout as exc:
                raise UpstreamError(f"Timed out fetching {label} from Yahoo Finance.") from exc
            except Exception as exc:  # yfinance raises a zoo of exception types
                # A 404 is definitive: fail fast instead of retrying with backoff.
                if "404" in str(exc) or "Not Found" in str(exc):
                    raise TickerNotFoundError(f"Ticker not found: {label}") from exc
                raise

        try:
            return attempt()
        except (TickerNotFoundError, UpstreamError):
            raise
        except Exception as exc:
            logger.warning("yfinance %s failed: %s", label, exc)
            raise UpstreamError(f"Yahoo Finance is unavailable or rate-limited ({label}). Try again
                shortly.") from exc
```

C4. Backend — the vectorised engine

The central design decision: every model function works on NumPy arrays where each row is one scenario. The base case is a batch of one row, the Monte Carlo a batch of 10,000 and the tornado a batch of 15 — so all three are guaranteed to use identical maths, and there is no Python loop over simulations. The projection:

backend/app/engine/dcf.py

```
def project_operations(
    *,
    revenue0: float,
    growth1: ArrayLike,
    terminal_growth: ArrayLike,
    ebitda_margin: ArrayLike,
    da_pct: ArrayLike,
    capex_pct: ArrayLike,
    nwc_pct: ArrayLike,
    tax_rate: ArrayLike,
    years: int,
    fade_years: int | None = None,
    terminal_capex_pct: ArrayLike | None = None,
) -> OperatingProjection:
    """Project revenue → EBITDA → EBIT → NOPAT → unlevered FCF for ``years`` years.

    Growth fades over ``fade_years`` (default ``years``, the DCF horizon); any years beyond it (needed
    when an LBO hold is longer than the DCF horizon) grow at terminal growth. Capex fades from
    ``capex_pct`` to ``terminal_capex_pct`` (default: no fade) over the same horizon and then stays there.
    """
    fade = fade_years or years
    t = np.arange(1, years + 1, dtype=float)[None, :]
    g1, gt = _col(growth1), _col(terminal_growth)
    growth = g1 + (gt - g1) * np.minimum(t - 1, fade) / fade
    revenue = revenue0 * np.cumprod(1 + growth, axis=1)
    ebitda = revenue * _col(ebitda_margin)
    d_and_a = revenue * _col(da_pct)
    ebit = ebitda - d_and_a
    taxes = np.maximum(ebit, 0.0) * _col(tax_rate)
    nopat = ebit - taxes
    c1 = _col(capex_pct)
    c_end = c1 if terminal_capex_pct is None else _col(terminal_capex_pct)
    capex_share = c1 + (c_end - c1) * (np.minimum(t - 1, fade - 1) / (fade - 1) if fade > 1 else 1.0)
    capex = revenue * capex_share
    prev_revenue = np.concatenate([np.full((revenue.shape[0], 1), revenue0), revenue[:, :-1]], axis=1)
    delta_nwc = _col(nwc_pct) * (revenue - prev_revenue)
    ufcf = nopat + d_and_a - capex - delta_nwc
    return OperatingProjection(revenue, growth, ebitda, d_and_a, ebit, taxes, nopat, capex, delta_nwc, ufcf)
```

Discounting and the bridge, with the mid-year convention handled exactly as described in B4:

backend/app/engine/dcf.py — value_dcf (core)

```
    n_years = ops.ufcf.shape[1]
    t = np.arange(1, n_years + 1, dtype=float)[None, :]
```

backend/app/engine/dcf.py — value_dcf (core) (continued)

```
    w = _col(wacc)
    g = _col(terminal_growth)[:, 0]
    periods = t - 0.5 if mid_year else t
    discount_factors = (1 + w) ** -periods
    pv_ufcf = ops.ufcf * discount_factors
    sum_pv = pv_ufcf.sum(axis=1)

    w1 = w[:, 0]
    final_fcf = ops.ufcf[:, -1]
    tv_perpetuity = final_fcf * (1 + g) / (w1 - g)
    tv_exit = np.atleast_1d(np.asarray(exit_multiple, dtype=float)) * ops.ebitda[:, -1]
    pv_tv_perpetuity = tv_perpetuity / (1 + w1) ** (n_years - 0.5 if mid_year else n_years)
    pv_tv_exit = tv_exit / (1 + w1) ** n_years

    ev_perpetuity = sum_pv + pv_tv_perpetuity
    ev_exit = sum_pv + pv_tv_exit
    adjust = -bridge.net_debt - bridge.minority_interest + bridge.non_operating_assets
    equity_perpetuity = ev_perpetuity + adjust
    equity_exit = ev_exit + adjust
```

The LBO debt schedule loops over *years* (at most seven), never over simulations; each line updates all 10,000 scenarios at once with vectorised minimum/maximum and conditional selection:

backend/app/engine/lbo.py — run_lbo (annual loop)

```
    for y in range(n_years):
        ebitda = ops.ebitda[:, y]
        i_s, i_sub, i_r = senior * rs, sub * rsub, revolver * rr
        interest = i_s + i_sub + i_r
        taxes = np.maximum(ops.ebit[:, y] - interest, 0.0) * tax
        fcf = ebitda - interest - taxes - ops.capex[:, y] - ops.delta_nwc[:, y]

        mandatory = np.minimum(senior, terms.senior_amortization * entry.senior0)
        excess = cash + fcf - entry.min_cash - mandatory
        draw = np.where(excess < 0, np.minimum(-excess, entry.revolver_capacity - revolver), 0.0)
        excess = excess + draw
        repay = np.where(excess > 0, np.minimum(revolver, excess), 0.0)
        excess = excess - repay
        sweep = np.where(excess > 0, np.minimum(senior - mandatory, terms.cash_sweep * excess), 0.0)
        excess = excess - sweep

        senior = senior - mandatory - sweep
        revolver = revolver + draw - repay
        cash = entry.min_cash + excess
        net_debt = senior + sub + revolver - cash
```

Correlated sampling is a few lines thanks to the Gaussian copula:

backend/app/engine/montecarlo.py

```
def sample(dists: dict[str, Distribution], keys: list[str], corr: np.ndarray, n: int, seed: int | None) ->
    dict[str, np.ndarray]:
    """Draw ``n`` correlated samples for each driver in ``keys``. Same seed => identical draws."""
    rng = np.random.default_rng(seed)
    z = rng.standard_normal((n, len(keys))) @ _factor(corr).T
    u = stats.norm.cdf(z)
    return {k: ppf(dists[k], u[:, j]) for j, k in enumerate(keys)}
```

And the glue that runs both models for any batch of driver values:

backend/app/engine/valuation.py

```
def evaluate(ctx: CompanyContext, v: dict[str, float], drivers: dict[str, np.ndarray], mid_year: bool) ->
    Evaluation:
    """Run DCF and LBO for a batch of stochastic driver values (each array shape (n,))."""
    dcf_years, terms = int(v["projection_years"]), lbo_terms(v)
    wacc = drivers["wacc"]
    tg_raw = drivers["terminal_growth"]
    tg = np.minimum(tg_raw, wacc - MIN_WACC_SPREAD)
    exit_multiple = np.maximum(drivers["exit_multiple"], 0.5)
    senior_rate = np.maximum(drivers["senior_rate"], 0.0)

    ops = project_operations(
        revenue0=ctx.revenue0, growth1=drivers["revenue_growth"], terminal_growth=tg,
            ebitda_margin=drivers["ebitda_margin"],
        da_pct=v["da_pct_revenue"], capex_pct=np.maximum(drivers["capex_pct_revenue"], 0.0),
```

backend/app/engine/valuation.py (continued)

```
            nwc_pct=v["nwc_pct_revenue"],
        tax_rate=v["tax_rate"], years=max(dcf_years, terms.exit_year), fade_years=dcf_years,
        terminal_capex_pct=v["terminal_capex_to_da"] * v["da_pct_revenue"],
    )
    dcf = value_dcf(ops.head(dcf_years), wacc=wacc, terminal_growth=tg, exit_multiple=exit_multiple,
        mid_year=mid_year, bridge=ctx.bridge)
    entry = build_entry(price=ctx.price, shares=ctx.bridge.shares, total_debt=ctx.bridge.total_debt,
        cash=ctx.bridge.cash,
                        revenue0=ctx.revenue0, ltm_ebitda=ctx.ltm_ebitda, terms=terms)
    lbo = run_lbo(
        ops.head(terms.exit_year), entry, terms, tax_rate=v["tax_rate"], senior_rate=senior_rate,
        sub_rate=senior_rate + (v["sub_rate"] - v["senior_rate"]), revolver_rate=senior_rate +
            (v["revolver_rate"] - v["senior_rate"]),
        exit_multiple=exit_multiple,
    )
    return Evaluation(ops, dcf, entry, lbo, terms, int((tg < tg_raw).sum()))
```

The tornado reuses that same function with a batch of 2k + 1 rows (each driver at its P10 and P90, plus the base case):

backend/app/engine/valuation.py — tornado

```
    drivers = {key: np.full(2 * k + 1, v[key]) for key in keys}  # last row = base case
    lows, highs = {}, {}
    for i, key in enumerate(keys):
        lows[key], highs[key] = quantile(dists[key], 0.10), quantile(dists[key], 0.90)
        drivers[key][2 * i], drivers[key][2 * i + 1] = lows[key], highs[key]
    ev = evaluate(ctx, v, drivers, settings.mid_year)
```

C5. Backend — assumptions and overrides

`resolve()` turns defaults plus user overrides into the final inputs. It validates keys and ranges, recomputes WACC when a component changes, moves the subordinated and revolver rates with the senior rate, enforces terminal growth below WACC and re-centres each distribution on its (possibly overridden) base value:

backend/app/engine/assumptions.py

```
def resolve(defaults: AssumptionSet, overrides: dict[str, float], distributions: dict[str, Distribution]) ->
    ResolvedAssumptions:
    """Apply user overrides, recompute WACC when its components change, and re-centre distributions."""
    unknown = set(overrides) - set(SPECS)
    if unknown:
        raise InvalidInputError(f"Unknown assumption keys: {', '.join(sorted(unknown))}", details={"keys":
            sorted(unknown)})
    unknown_d = set(distributions) - set(STOCHASTIC_KEYS)
    if unknown_d:
        raise InvalidInputError(f"Distributions are only supported for: {', '.join(STOCHASTIC_KEYS)}",
            details={"keys": sorted(unknown_d)})

    base = {a.key: a.value for a in defaults.items}
    values = {**base, **overrides}
    for key, v in overrides.items():
        s = SPECS[key]
        if not s.lo <= v <= s.hi:
            raise InvalidInputError(f"{s.label} must be between {s.lo} and {s.hi}.", details={"key": key,
                "value": v})

    if "wacc" not in overrides and any(k in overrides for k in WACC_COMPONENTS):
        ke = cost_of_equity(values["risk_free"], values["beta"], values["equity_risk_premium"])
        values["wacc"] = wacc(ke, values["cost_of_debt"], values["tax_rate"], values["debt_weight"])
    # The sub-debt and revolver rates are defined as spreads over the senior rate unless set explicitly.
    if "senior_rate" in overrides:
        for k in ("sub_rate", "revolver_rate"):
            if k not in overrides:
                values[k] = base[k] + (values["senior_rate"] - base["senior_rate"])
    for k in ("projection_years", "exit_year"):
        values[k] = float(round(values[k]))

    if values["terminal_growth"] >= values["wacc"] - 0.005:
        raise InvalidInputError("Terminal growth must be at least 0.5pp below WACC.",
                                details={"wacc": values["wacc"], "terminal_growth": values["terminal_growth"]})

    dists: dict[str, Distribution] = {}
```

backend/app/engine/assumptions.py (continued)

```
    by_key = {a.key: a for a in defaults.items}
    for key in STOCHASTIC_KEYS:
        if key in distributions:
            dists[key] = distributions[key]
        else:
            default = by_key[key].distribution
            assert default is not None
            dists[key] = shift_distribution(default, base[key], values[key]) if values[key] != base[key]
                else default
    return ResolvedAssumptions(values=values, distributions=dists)
```

C6. Backend — API and errors

Routes are thin: validate, rate-limit, delegate to the service. Handlers are synchronous functions, so FastAPI runs the blocking data calls in its thread pool.

backend/app/api/routes.py

```
@router.post("/valuation", response_model=ValuationResponse, responses={404: {"model": ErrorResponse}},
    tags=["valuation"])
@limiter.limit(settings.rate_limit_valuation)
def valuation(request: Request, body: ValuationRequest, service: CompanyService = Depends(get_service)) ->
    ValuationResponse:
    resp, _ = service.valuation(body)
    return resp
```

Every failure becomes the same JSON shape, `{"error": {"code", "message", "details"}}`:

Code	HTTP	When
`ticker_not_found`	`404`	Yahoo has no such symbol
`invalid_ticker`	`422`	The symbol contains invalid characters
`validation_error`	`422`	The request body fails schema validation
`invalid_input`	`422`	An override is out of range, or terminal growth ≥ WACC − 0.5%
`correlation_not_psd`	`422`	The correlation matrix is invalid (details include a suggested fix)
`financial_company`	`422`	DCF/LBO requested for a bank, insurer, NBFC or REIT
`insufficient_data`	`422`	The data-quality checks failed
`upstream_unavailable`	`502`	Yahoo Finance timed out or rate-limited after retries
`rate_limited`	`429`	Too many requests from one client

Endpoint	Purpose
`GET /api/health`	Liveness and provider name
`GET /api/search?q=`	Company and ticker autocomplete
`GET /api/company/{ticker}`	Profile, cleaned financials, data quality, classification, suitability, default assumptions
`POST /api/valuation`	Full valuation (DCF, LBO, Monte Carlo, sensitivity, comps, football field)
`POST /api/export/excel`	Same body → .xlsx workbook
`POST /api/export/memo`	Same body → one-page PDF

C7. Frontend

Typed API client

Every response is parsed with a Zod schema that mirrors the Pydantic models, so a contract change fails loudly instead of rendering wrong numbers. Search uses a hand-written check instead, which keeps Zod out of the bundle that every page loads:

frontend/src/lib/api.ts

```
async function getJson<T extends z.ZodType>(schema: T, path: string, init?: RequestInit): Promise<z.infer<T>> {
  const res = await request(path, init);
  const parsed = schema.safeParse(await res.json());
  if (!parsed.success) {
    throw new ApiError("The API returned an unexpected response shape.", "schema_mismatch", res.status,
        parsed.error.issues);
  }
  return parsed.data;
```

Workspace state, debouncing and re-runs

The workspace keeps overrides, distributions, settings and peers in React state, derives the request from them, debounces it, and lets TanStack Query fetch — keeping the previous result on screen while the new one runs:

frontend/src/components/analyze/workspace.tsx

```
  const request = React.useMemo<ValuationRequest>(
    () => ({ ticker: company.profile.ticker, overrides, distributions, settings, peers }),
    [company.profile.ticker, overrides, distributions, settings, peers],
  );
  const debounced = useDebounced(request, DEBOUNCE_MS);
  const pendingChange = debounced !== request;

  const valuationQ = useQuery({
    queryKey: ["valuation", debounced],
    queryFn: ({ signal }) => api.valuation(debounced, signal),
    placeholderData: keepPreviousData,
    retry: false,
```

Share links

The share codec stores only what differs from the defaults, as base64url-encoded JSON, and validates it with Zod when decoding:

frontend/src/lib/share.ts

```
export function encodeShare(state: ShareState, defaults: SimSettings): string {
  const s: Partial<SimSettings> = {};
  (Object.keys(state.settings) as (keyof SimSettings)[]).forEach((k) => {
    if (state.settings[k] !== defaults[k]) (s as Record<string, unknown>)[k] = state.settings[k];
  });
  const payload = { o: state.overrides, d: state.distributions, s, p: state.peers };
  const empty = !Object.keys(state.overrides).length && !Object.keys(state.distributions).length &&
      !Object.keys(s).length && !state.peers;
  return empty ? "" : toBase64Url(JSON.stringify(payload));
}

export function decodeShare(encoded: string | null, defaults: SimSettings): ShareState | null {
```

Charts

Charts read colours from the design tokens (CSS variables), so they follow the theme automatically. Histograms are pre-binned on the backend; the browser draws them with Recharts. The football field and tornado are custom SVG (RangeBars) so that labels can be placed pixel-accurately; their bars carry their final width in the markup and CSS only animates the reveal, so they render correctly in exports, screenshots and without JavaScript.

Themes

Each palette is a block of CSS variables keyed by a data attribute. Choosing a theme sets attributes on the <html> element; a tiny script restores saved choices before the first paint so there is no flash of the default theme:

frontend/src/app/themes.css (one palette, light)

```
[data-palette="monaco"] {
  --bg: #f4f7f2;
  --bg-elevated: #ffffff;
  --surface: #ffffff;
```

frontend/src/app/themes.css (one palette, light) (continued)

```
  --surface-2: #eaf0e8;
  --surface-3: #dbe5d8;
  --border: #d9e3d6;
  --border-strong: #b9c9b4;
  --fg: #0b1f16;
  --fg-muted: #3a5244;
  --fg-subtle: #52665a;
  --accent: #0b5d3b;
  --accent-strong: #084a2f;
  --accent-fg: #ffffff;
  --accent-soft: rgb(11 93 59 / 0.1);
  --positive: #15803d;
  --negative: #b91c1c;
  --warning: #8a6d00;
  --series-dcf: #0b5d3b;
  --series-lbo: #8a6d00;
  --series-comps: #1d4ed8;
  --series-market: #55695d;
  --marker: #b91c1c;
  --grid: rgb(11 31 22 / 0.07);
```

frontend/src/lib/appearance.ts

```
export function applyAppearance(a: Appearance) {
  const el = document.documentElement;
  for (const k of KEYS) el.setAttribute(`data-${k}`, a[k]);
  syncDarkClass();
  try {
```

C8. Exports

The Excel DCF tab writes formulas, not numbers. For example, revenue in each forecast year references the previous year and the growth row, and the growth row implements the fade formula from B4 directly against the Assumptions tab:

backend/app/export/excel.py

```
        formulas = {
            5: (f"={p}5+1", FMT_INT),
            6: (f"={g1}+({tg}-{g1})*({c}5-1)/{N}", FMT_PCT),
            7: (f"={p}7*(1+{c}6)", FMT_MONEY),
            8: (f"={c}7*{a['ebitda_margin']}", FMT_MONEY),
            9: (f"={c}7*{a['da_pct_revenue']}", FMT_MONEY),
            10: (f"={c}8-{c}9", FMT_MONEY),
            11: (f"=MAX({c}10,0)*{a['tax_rate']}", FMT_MONEY),
            12: (f"={c}10-{c}11", FMT_MONEY),
            13: (f"={c}7*({cx1}+({a['terminal_capex_to_da']}*{a['da_pct_revenue']}-{cx1})*({c}5-1)/MAX({N}-1
                ,1))", FMT_MONEY),
            14: (f"={a['nwc_pct_revenue']}*({c}7-{p}7)", FMT_MONEY),
            15: (f"={c}12+{c}9-{c}13-{c}14", FMT_MONEY),
            16: (f"={c}5-0.5*{a['mid_year']}", '0.0'),
            17: (f"=1/(1+{a['wacc']})^{c}16", FMT_FACTOR),
            18: (f"={c}15*{c}17", FMT_MONEY),
```

The memo is built with ReportLab. It registers a Unicode font when one is available so ₹ renders, draws the football field as vector graphics, and generates the risk bullets and the plain-language framing from the results.

C9. Testing

File	What it proves
`test_dcf.py`	Projection and valuation match the Acme hand calculation to the cent, with and without mid-year; bridge; capex fade.
`test_lbo.py`	Sources = uses; debt schedule, sweep, revolver draws, covenant flags; IRR and MOIC to six decimals; infeasible cases.
`test_montecarlo.py`	Each distribution has the requested moments; truncation bounds; correlation recovered; non-PSD rejected; same seed → identical results; 10,000 runs in under 2 seconds; tornado ordering.

File	What it proves
`test_properties.py`	Economics: higher WACC → lower value; higher exit multiple → higher IRR; higher margin → higher value; higher rates or premium → lower IRR.
`test_suitability.py`	Every sub-score against hand calculation; weight redistribution; classification; P/B–ROE and DDM.
`test_normalize.py`	Label aliases, fallbacks, dropped periods, quality flags, default assumptions, overrides, cache, Indian formatting.
`test_api.py`	Every endpoint end to end with an in-memory fake provider (no network), including exports and error codes.

Run with `npm test`. The frontend is checked with ESLint and a strict TypeScript compile (`npm run lint`).

C10. Extending Rolletto

Task	Where and how
Use a paid data feed	Write a class implementing DataProvider (provider.py) that returns the same schemas, and return it from get_service() in api/deps.py. Nothing else changes.
Add a stochastic driver	Add it to SPECS and STOCHASTIC_KEYS in engine/assumptions.py with a default distribution, use drivers[key] in evaluate(), and add its label to the correlation table in the Monte Carlo tab.
Change country defaults	Edit the table in core/config.py, or set ROLLETTO_COUNTRY_OVERRIDES as JSON without touching code.
Add a theme	Add a light and a dark block to app/themes.css and an entry to PALETTES and PRESETS in lib/appearance.ts.
Add a peer list	Add the industry and tickers to INDIA_PEERS in data/peers.py.

C11. Limitations

- yfinance is unofficial: fields can be missing, late or wrong, and Yahoo rate-limits. A first request for a new company can take a few seconds.
- Annual statements only; stock-based compensation, leases, pensions, tax-loss carry-forwards and dilution from options are not modelled separately.
- Diluted shares are approximated as market capitalisation ÷ price.
- The LBO assumes one exit and no interim dividends, and ignores limits on interest deductibility.
- Automatic peers may need editing; they come from Yahoo's industry and sector leaders or curated Indian lists.
- The distributions describe uncertainty in your inputs — not every way the real world can surprise.

Glossary

Term	Meaning
DCF	Discounted cash flow — value as the present value of future free cash flows.
UFCF	Unlevered free cash flow — cash generated by operations before any debt payments.
WACC	Weighted average cost of capital — the blended return required by debt and equity holders.
Terminal value	The value of all cash flows beyond the forecast horizon.
EV	Enterprise value — the value of the operating business to all capital providers.
LBO	Leveraged buyout — an acquisition funded mostly with debt, repaid from the company's cash flow.
IRR / MOIC	Internal rate of return / multiple of invested capital — the sponsor's annualised and total return.
Covenant	A lender's condition, e.g. net debt must stay below 6x EBITDA.
Monte Carlo	Estimating a distribution by simulating many random scenarios.

Term	Meaning
Copula	A way to join individual distributions with a chosen correlation structure.
Football field	A chart comparing value ranges from different methods on one axis.
Tornado chart	A sensitivity chart showing how much each input moves the output.
P10 / P50 / P90	The 10th, 50th (median) and 90th percentiles of a distribution.

Rolletto is an educational tool. Nothing in this guide or the application is investment advice.